

Study_Recorder_Program Document (v1.0)

- CommonUtils

1. MySQL structure

I. Term

Id	Name (such as Spring 2024)	Start_date
Primary key, int, auto increment, not null	char(15)	Date default curdate()

II. Courses

Id	Name	Description	Term_id	Major_related
Primary key, int, auto increment, not null	Char(16)	text	int	bool

III. Note

Id	Course_id	Content	File_name (each note contains at most 1 file)	Time_added	Time_updated
Primary key, int, auto	int	text	Varchar(40)	Timestamp DEFAULT CURRENT_TIMESTAMP	DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

increment,					
not null					

Code:

```

create table terms (
  id int primary key not null auto_increment,
  name char(15),
  start_date date default (CURRENT_DATE)
);
create table courses (
  id int primary key not null auto_increment,
  name varchar(20),
  description text,
  term_id int,
  in_progress bool
  major_related bool
);
create table note (
  id int primary key not null auto_increment,
  course_id int,
  content text,
  file_name varchar(50),
  time_added Timestamp DEFAULT CURRENT_TIMESTAMP,
  time_updated DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
)
create table users (
  id int primary key auto_increment not null,
  username varchar(20) not null,
  password varchar(45) not null
)

```

2. Pojos

Term, Course, Note

- Client End (on port 80)

Layers

I. Dao interfaces (**MyBatis** has corresponding mappers for each method)

- TermDAO

List<Term> getAllTerms(); // by default, sort by term start date

- CourseDAO

List<Course> getAllCourses(); // by default, sort by term start date and then course name

List<Course> getCoursesByTerm(int termId); // by default, sort by and then course name

- NoteDAO

List<Note> getNotesByTermId(int termId); // by default, sort by note created time

List<Note> getNotesByCourseId(int courseId); // by default, sort by note created time

II. Service

i. TermService

List<Term> getTerms();

ii. CourseService

List<Course> getCourses();

List<Course> getCoursesByTerm(Term term);

// the methods below will require authentication, and sanctions

may be placed on those ip's who abuse it (interceptors are used)

Boolean addCourse(Course course);

Boolean editCourse(Course course);

Boolean deleteCourseById(int courseId);

iii. NoteService

List<Note> getNotes(); // gets all notes; by default notes are sorted by time added

```
List<Note> getNotesByCourseId(int courseId);
```

```
// the methods below will require authentication, and sanctions
```

```
may be placed on those ip's who abuse it (interceptors are used)
```

```
Boolean addNote(Note note);
```

```
Boolean editNote(Note note);
```

```
Boolean deleteNoteById(int noteId);
```

3. Interceptor

Deny access to those who are on the blacklist && and list those who causes more than 3 exceptions to my server to the list.

Study_Recorder_Program Document (v1.0)

- Administrator End (on port 10588)

1. Service layer

// the methods below will require authentication, and sanctions may be placed on those ip's who abuse it (interceptors are used)

Boolean addTerm(Term term);

Boolean editTerm(Term term);

Boolean deleteTermById(int termId); // delete all corresponding courses and notes

Boolean addCourse(Course course);

Boolean editCourse(Course course);

Boolean deleteCourseById(int courseId); // delete all corresponding notes

Boolean addNote(Note note);

Boolean editNote(Note note);

Boolean deleteNoteById(int noteId);

2. Interceptor

Place those who try to breach the system on the blocklist.

a. Blocklist

Achieved by adding those ip's to the blocklist (on Redis) that are denied by the authenticator 3 times.

b. Deny access to those who are on the blocklist.

MySQL username: leon password: vaqneb-Rucgar-turfe6

Tomcat username: 1Yu8v password: m6St4hgqTC

Redis only allow local access (bind local && yes to protected mode)

Interceptors:

1. Check ip query quota (200 per day) -Associated with **remote address**
Redis: key-value, expire time one day(add the option NX -- Set expiry only when the key has no expiry)
Key: in format of "ip-x.x.x.x"
Value: 199 if newly set, and decrement by 1 if value is greater than 1 when querying.
2. Check login info -Associated with **sessionId**
Redis: key-value, expire time 3 hours
Key: in format of "sid-x...x" (sid for sessionId)
Value: only store value "yes" if they is logged in. Set the expiry of 3 hours the first time the session is authenticated.